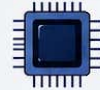
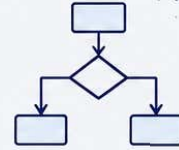


1010101010
0101010101
1010101010
0101010101



Infographics



Visual Learning

Better understanding through visual content.



Simplified Concepts

Complex topics explained in simple and visual form.



Quick Revision

Easy to revise important points and diagrams.

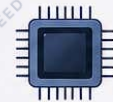
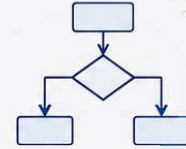


Exam Ready

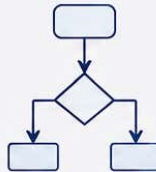
Helps in faster revision and better exam preparation.



1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Introduction to Software Engineering

Characteristics of Software

"Software is not a physical product, but a collection of programs, procedures, rules and documentation that performs a task."



What is Software ?

Software is a set of **programs**, **procedures**, associated **documentation** and **data** that directs a computer system to perform specific tasks and solve problems for users.

Software vs Hardware

Software	Hardware
Intangible (not physical)	Tangible (physical)
Developed/engineered	Manufactured
Does not wear out	Wears out over time
Easy to modify	Difficult to change
Copying is easy	Copying is not possible

Importance

- ✓ Controls hardware resources
- ✓ Provides user interface
- ✓ Helps in solving real world problems
- ✓ Used in almost every field
- ✓ Drives modern technologies (AI, IoT, Cloud, etc.)

Key Characteristics of Software

1 Intangible



- Software is not a physical entity, it cannot be touched or seen.
- It exists as code, documentation and data.

2 Developed, not Manufactured



- Software is developed through engineering processes.
- It is not manufactured like hardware, so no physical production is involved.

3 Does Not Wear Out



- Unlike hardware, software does not wear out physically.
- However, it can become obsolete due to changes in user requirements or technology.

4 Highly Complex



- Modern software systems contain millions of lines of code.
- Complex due to multiple features, dependencies and changing requirements.

5 Easy to Modify



- Software can be easily modified to meet new requirements.
- Changes can be made without rebuilding the entire system.

6 Copying is Easy



- Software can be copied quickly and at low cost.
- Identical copies can be distributed worldwide.

7 Conforms to Requirements



- Software is built according to specific user requirements.
- It must meet functional and non-functional requirements.

8 Largely Custom-Built



- Most software is developed for specific needs of users or organizations.
- Even general-purpose software is often configured for users.

9 Continuously Evolving



- Software needs regular updates to fix bugs, add features and adapt to new environments.
- Evolution is a continuous process throughout its life cycle.

10 Not Affected by Environment



- Software is not directly affected by environmental factors like temperature, humidity, etc.
- But it may become incompatible with new hardware or platforms.

Key Takeaway :

Software is unique because it is intangible, complex, easy to modify and does not wear out, but it must evolve continuously to meet changing user needs.

Better Software
A Better Tomorrow

"Different types of software work together to make a computer system useful and powerful."



Types of Software

Software can be broadly classified into different types based on their purpose, usage and functionality.

"Right software for the right task brings better results."



1 System Software

System software manages the hardware resources of a computer system and provides a platform for application software to run.

Examples and Components:



Operating System

(Windows, Linux, macOS, Android)



Device Drivers

(Printer driver, Graphics driver)



Utility Programs

(Antivirus, Disk Cleanup, Backup tools)



Firmware

(BIOS/UEFI)

Key Point:

- Acts as an interface between user and hardware.
- Essential for the functioning of a computer system.

4 Middleware

Middleware acts as a bridge between different applications, systems or databases, allowing them to communicate and work together.



Examples:

- Application servers (Tomcat, JBoss)
- Database middleware (ODBC, JDBC)
- Message-oriented middleware (RabbitMQ)

Conclusion:

Each type of software has a specific role, and together they make the computer system versatile, powerful and useful for various user needs.

2 Application Software

Application software is designed to perform specific tasks for users. It runs on top of system software and helps in solving real-world problems.

Examples:



Word Processors (MS Word, Google Docs)



Spreadsheets (MS Excel, Google Sheets)



Presentation Software (MS PowerPoint)



Web Browsers (Chrome, Firefox, Edge)



Media Players (VLC Media Player)



Graphics Software (Photoshop, CorelDRAW)



Email Clients (Outlook, Thunderbird)

Key Point:

- Designed to meet user needs.
- Can be general-purpose or specific-purpose.

5 Embedded Software

Embedded software is designed to perform specific functions within dedicated hardware devices. It is an integral part of the device.



Examples:

- Software in smartphones
- Software in home appliances (washing machines, AC)
- Software in cars (engine control systems)
- Software in medical devices (ECG machines)

3 Programming Software

Programming software provides tools and environment to develop, test and debug other software programs.

Examples:



Compilers (GCC, javac)



Interpreters (Python, JavaScript)



Assemblers (Convert assembly code to machine code)



IDEs (Eclipse, IntelliJ IDEA, VS Code)



Debugging Tools (GDB, Debugger in IDEs)



Other Development Tools

(Build tools, Version control tools like Git)

Key Point:

- Used by software developers.
- Helps in creating other software.

6 Web Software

Web software runs on web servers and is accessed through a web browser over a network (usually the internet).



Examples:

- Web applications (Gmail, Google Drive)
- E-commerce websites (Amazon, Flipkart)
- Online learning platforms (Coursera, Udemy)
- Social media platforms (Facebook, Instagram)

"Software
Drives the Digital World"

"Good software is not an accident, it is the result of systematic engineering."

Need of Software Engineering

"Engineering principles turn ideas into reliable software."



Software Engineering is needed to develop **high-quality**, reliable and **maintainable** software systems that meet user needs in a cost-effective and timely manner.

Why is Software Engineering Needed ?

1 Increasing Complexity



Modern software systems are large and complex. Software engineering provides systematic methods to manage this complexity.

2 Growing Size of Software



Today's software systems contain millions of lines of code. Software engineering helps in proper planning, design and management.

3 Changing Requirements



User needs and business environments keep changing. Software engineering provides flexible processes to handle changing requirements.

4 Need for High Quality



Software must be reliable, secure, efficient and user-friendly. Software engineering ensures quality through standards, reviews and testing.

5 Cost and Time Constraints



Software projects must be delivered on time and within budget. Software engineering provides estimation, planning and project management techniques.

6 Team Development



Large software projects are developed by teams. Software engineering provides methods, standards and documentation to improve collaboration.

7 Maintainability



Software must be updated and maintained for a long time. Software engineering helps in writing modular and well-documented code.

8 Reusability



Software engineering promotes reuse of existing components, saving time and cost, and improving reliability.

9 Handling Risks



Software projects face technical, schedule, cost and operational risks. Software engineering provides risk management techniques to identify and minimize risks.

10 Wide Application Domain



Software is used in almost every field - education, healthcare, banking, transportation, defense, entertainment, etc. Software engineering is needed to develop reliable systems for these critical applications.

Key Takeaway



Software engineering is necessary to develop software that is reliable, maintainable, cost-effective and meets the needs of users in a changing world.

Real-World Examples



Hospital Management



Banking Systems



Airline Reservation



E-commerce Websites



Mobile Applications

Conclusion

Without software engineering large and reliable software systems would be very difficult, costly and risky to develop and maintain in today's world.

"Software Engineering makes better software for a better tomorrow."



"Plan well,
Build well,
Deliver better."

Software Development Life Cycle (SDLC)

A systematic process followed to develop high-quality software that meets user requirements in a structured, cost-effective and timely manner.

"SDLC turns ideas into useful software through a step-by-step process."



1 Requirement Analysis

- Understand the problem and user needs.
- Collect and analyze requirements (functional and non-functional).
- Define scope, feasibility and document requirements.



Output: Software Requirement Specification (SRS)

3 Implementation (Coding)

- Develop the software as per design.
- Write, review and integrate code.
- Follow coding standards and best practices.
- Perform unit testing.



Output: Source Code (Programs)

5 Deployment

- Release the software to the production environment.
- Install, configure and make it available to users.
- Provide user training and support.



Output: Live Software System

Key Features of SDLC

- Structured and systematic approach
- Clear phases and deliverables
- Improves quality and reliability
- Helps in cost and time estimation
- Better project management

Importance of SDLC

- Ensures better planning and control
- Reduces risks and rework
- Improves communication among stakeholders
- Delivers high-quality software
- Increases customer satisfaction
- Helps in meeting deadlines and budget

Common SDLC Models

- Waterfall Model
- V-Model
- Incremental Model
- Iterative Model
- Spiral Model
- Agile Model
- DevOps Model

Real-World Example



Banking System



E-commerce Website



Mobile App

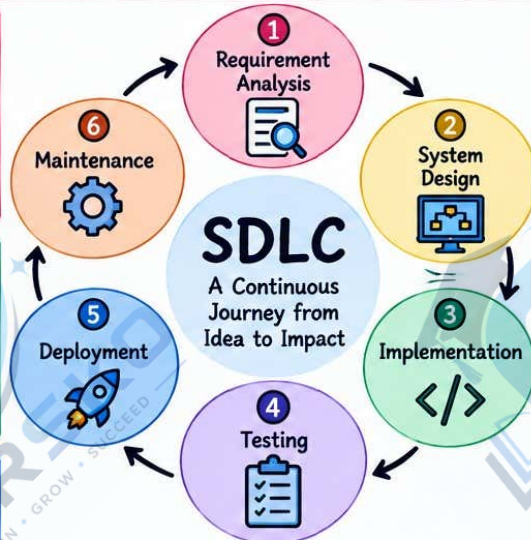


Hospital Management System

Key Takeaway:

SDLC provides a well-defined path from user requirements to a working software product, ensuring quality, efficiency and customer satisfaction.

"Good Process
Builds Great Software."



Plan → Design → Build → Test → Deploy → Maintain

Better Software 🌱 A Better Tomorrow

2 System Design

- Convert requirements into a technical solution.
- Design system architecture, database, user interface and modules.
- Choose appropriate technologies.



Output: System Design Document (SDD)

4 Testing

- Check whether the software meets requirements.
- Find and fix defects (bugs).
- Types: Unit testing, Integration testing, System testing, Acceptance testing.



Output: Tested and Bug-free Software

6 Maintenance

- Fix issues after deployment.
- Update software to meet changing needs.
- Improve performance and add new features.
- Types: Corrective, Adaptive, Perfective, Preventive maintenance.



Output: Updated and Improved Software

"A well-defined process leads to a successful product."

Phases of SDLC

The Software Development Life Cycle (SDLC) consists of a series of phases that guide the development of software from initial idea to final deployment and maintenance.

"Plan → Build
→ Test → Deliver
→ Improve"
That's SDLC!

1

Requirement Analysis



Understand the problem, user needs and collect requirements.

Key Activities:

- Identify stakeholders
- Gather requirements
- Analyze requirements (functional & non-functional)
- Define scope and feasibility

Output:

Software Requirement Specification (SRS)

2

System Design



Convert requirements into a technical solution (architecture, design, database, UI).

Key Activities:

- Design system architecture
- Database design
- UI/UX design
- Select technologies
- Prepare design documents

Output:

System Design Document (SDD)

3

Implementation (Coding)



Develop the software as per design using a programming language.

Key Activities:

- Write source code
- Follow coding standards
- Use version control (e.g., Git)
- Perform unit testing

Output:

Source Code (Programs)

4

Testing



Check whether the software meets the requirements and is free from defects.

Key Activities:

- Prepare test cases
- Perform unit, integration, system and acceptance testing
- Find and fix defects
- Ensure quality and reliability

Output:

Tested and Bug-free Software

5

Deployment



Release the software to the production environment and make it available to users.

Key Activities:

- Install and configure software
- Migrate data (if needed)
- Provide user training
- Go live
- Monitor initial performance

Output:

Live Software System

6

Maintenance



Fix issues, update features and ensure the software continues to work effectively.

Key Activities:

- Corrective maintenance (fix bugs)
- Adaptive maintenance (adjust to new environment)
- Perfective maintenance (improve performance)
- Preventive maintenance (avoid future issues)

Output:

Updated and Improved Software

Key Takeaways



- SDLC is a step-by-step process.
- Each phase has a specific goal and deliverables.
- Helps in building high-quality, reliable and maintainable software.
- Used in all types of software projects (small or large).

Benefits of Following SDLC

- ✓ Better planning and control
- ✓ Improved quality
- ✓ Reduced risks and rework
- ✓ Timely delivery
- ✓ Higher customer satisfaction



Final Outcome

"A systematic SDLC leads to better software and a better tomorrow."

Plan Today... Build a Better Tomorrow!



"A development model is like a roadmap that guides the software project to success."

Software Development Models : Introduction

Software Development Models are structured approaches or frameworks that define the process, activities, flow and control of software development from initial idea to final product.

Different projects need different paths. Choose the right model for better results!



What is a Software Development Model ?

A software development model is a predefined process that defines a sequence of activities, tasks, milestones and deliverables to develop, test, deploy and maintain software.



"It provides a structure to plan, manage and control the software development process."

Objectives



- ✓ Provide a systematic approach
- ✓ Improve product quality
- ✓ Manage project risks
- ✓ Ensure timely delivery
- ✓ Optimize resource utilization
- ✓ Increase customer satisfaction

Need for Development Models

- ✓ Handle complexity of large projects
- ✓ Provide clear structure and guidance
- ✓ Help in project planning and estimation
- ✓ Manage risks and changes
- ✓ Improve communication among stakeholders
- ✓ Ensure better quality and maintenance



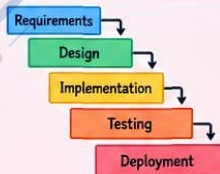
Factors for Choosing a Model

- ✓ Project size and complexity
- ✓ Clarity of requirements
- ✓ Client involvement
- ✓ Time and budget constraints
- ✓ Risk level
- ✓ Technology and domain
- ✓ Need for maintenance



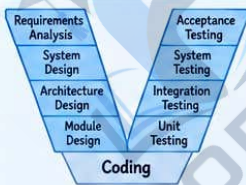
Common Software Development Models

1 Waterfall Model



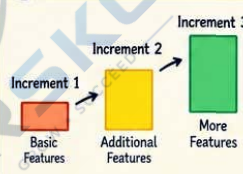
A linear and sequential model. Each phase must be completed before the next phase begins.

2 V-Model



An extension of waterfall model. Each development phase has a corresponding testing phase.

3 Incremental Model



The software is developed in small, manageable increments. Each increment adds new functionality to the system.

4 Iterative Model



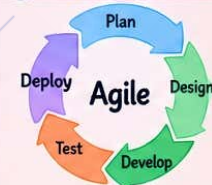
Development is done in iterations. Each iteration goes through the complete development cycle, allowing continuous improvements.

5 Spiral Model



A risk-driven model that combines iterative development with systematic risk analysis.

6 Agile Model



A flexible and adaptive model based on iterative development, collaboration and customer feedback.

Key Points

- A development model provides a structured approach.
- There is no one-size-fits-all model.
- Choice depends on project requirements, risk and constraints.
- Modern projects often use hybrid models (e.g., Agile + Waterfall).
- The goal is to deliver high-quality software efficiently.

Comparison (At a Glance)

Model	Nature	Flexibility	Customer Involvement	Suitable For
Waterfall	Linear	Low	Low	Small, well-defined projects
V-Model	Linear	Low	Low	Safety-critical systems
Incremental	Step-by-step	Medium	Medium	Medium to large projects
Iterative	Repeated cycles	High	Medium	Evolving requirements
Spiral	Risk-driven	Medium	High	Large, complex, high-risk projects
Agile	Iterative & Flexible	High	High	Dynamic, changing requirements

Conclusion

Software development models provide a roadmap for successful software development. Selecting the right model helps in managing risks, ensuring quality, meeting customer needs and delivering software on time and within budget.



"Right Model + Right Approach = Successful Software"

"A process model provides a roadmap for successful software development."

Introduction to Process Models

"Different paths can lead to the same destination — choose the model that fits your needs."

A process model is a structured framework that defines the sequence of activities, tasks, milestones, roles and deliverables involved in developing, delivering and maintaining software.

1 What is a Process Model ?

- A process model is a simplified representation of the software development process.
- It defines the order in which activities are performed.
- It provides a framework to plan, manage and control the development process.
- It acts as a guideline for building high-quality software within time and budget.



"A process model tells us WHAT to do, WHEN to do it, and HOW to do it."

2 Characteristics of Process Models

- ✓ Provides a structured approach
- ✓ Defines clear phases and activities
- ✓ Specifies inputs, outputs and deliverables
- ✓ Helps in project planning and estimation
- ✓ Provides a basis for monitoring and control
- ✓ Supports risk management
- ✓ Can be adapted according to project needs
- ✓ Aims to deliver quality software



3 Objectives of Process Models

- ✓ Ensure systematic development
- ✓ Improve product quality
- ✓ Manage project risks
- ✓ Provide better communication among stakeholders
- ✓ Help in resource allocation
- ✓ Enable progress tracking and control
- ✓ Deliver software on time and within budget
- ✓ Support maintenance and future enhancement



4 Why Process Models are Needed ?

- ✓ Software development is a complex activity.
- ✓ Large projects require planning and coordination.
- ✓ Ensures consistent and organized development.
- ✓ Helps in identifying and managing risks.
- ✓ Improves productivity and quality.
- ✓ Provides a common understanding among team members.
- ✓ Helps in cost and time estimation.
- ✓ Useful for monitoring and evaluation.

5 Common Types of Process Models

- Waterfall Model
- V-Model
- Incremental Model
- Iterative Model
- Spiral Model
- Agile Model
- Prototype Model
- RAD (Rapid Application Development)
- DevOps Model



Different models for different needs

6 General Process Model Framework



7 Real-World Analogy



Just like a building requires a proper plan, design, construction and inspection, software also needs a process model to ensure it is built correctly and serves its purpose.

8 Key Takeaways

- ✓ A process model provides a roadmap for software development.
- ✓ There is no single best model for all projects.
- ✓ Choice of model depends on project requirements, team size, budget, risk and customer needs.
- ✓ Process models help in delivering high-quality software.
- ✓ Modern projects often use a combination of models (e.g., Agile + DevOps).
- ✓ The ultimate goal is to satisfy user needs efficiently and effectively.



Plan
Execute
Deliver
Improve

9 Conclusion

Process models are essential in software engineering as they provide a structured and systematic approach to develop, deliver and maintain software. By selecting the right model, organizations can reduce risks, improve quality, meet deadlines and achieve customer satisfaction.



"Right Process → Better Management → High-Quality Software → Happy Users"

Choose Smart.
Develop Better.
Build the Future.

A good process model doesn't just build software, it builds success.

"Plan well,
proceed step by step,
reach the destination."

Introduction to Waterfall Model

"One step
at a time,
from requirements
to a working system."



The Waterfall Model is a linear and sequential software development model in which each phase is completed before moving to the next phase. It is one of the earliest and simplest process models.

1 What is Waterfall Model ?

The Waterfall Model is a traditional software development model that follows a linear and sequential approach. Each phase produces a specific deliverable and must be completed before the next phase begins. It is also known as a "Plan-Driven Model" because extensive planning is done before development starts.



"In Waterfall, progress flows in one direction
— like a real waterfall."

5 Advantages



- ✓ Simple and easy to understand.
- ✓ Clear structure with well-defined phases.
- ✓ Extensive documentation.
- ✓ Easy to manage and control.
- ✓ Works well for small and medium projects.
- ✓ Progress is measurable (each phase has deliverables).

8 Waterfall Model in a Nutshell

The Waterfall Model follows a "do it once, do it right" approach. It is best suited for projects with clear, stable requirements and is widely used in traditional industries such as banking, government, and education.



2 Phases of Waterfall Model

1. Requirements Analysis

Understand and document user requirements.

2. System Design

Design the system architecture and components.

3. Implementation

Write and integrate the code as per design.

4. Testing

Verify and validate the software.

5. Deployment

Release the software to users.

6. Maintenance

Fix issues and make improvements after deployment.

6 Disadvantages



- ✗ Not suitable for large and complex projects.
- ✗ Changes are difficult and costly.
- ✗ Working software is available only at the end.
- ✗ High risk if requirements are not clear.
- ✗ No overlap between phases (rigid model).
- ✗ Less flexible compared to modern models.

9 Key Takeaways

- ✓ Waterfall Model is a linear and sequential process model.
- ✓ Each phase must be completed before the next begins.
- ✓ Best for stable and well-defined requirements.
- ✓ Simple, structured and easy to manage.
- ✓ Not flexible to changes.

3 Characteristics

- ✓ Linear and sequential approach
- ✓ Each phase has specific deliverables
- ✓ Extensive documentation
- ✓ Simple and easy to understand
- ✓ Well-defined structure
- ✓ Suitable for small and medium-sized projects
- ✓ Changes are difficult once a phase is completed
- ✓ Works well when requirements are clear and stable



4 When to Use Waterfall Model ?

- When project requirements are well understood.
- When requirements are stable and less likely to change.
- For small and medium-sized projects.
- In projects with low risk and clear scope.
- When the technology is well known.



7 Real-World Examples



Simple Desktop Applications



Payroll Systems



Library Management System



Student Information System



Small Business Applications

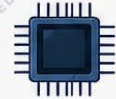
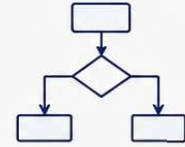
Conclusion

The Waterfall Model is a foundational software development model that provides a clear and structured approach to build software. Although it lacks flexibility, it is still useful for projects with well-defined and stable requirements.



"A clear path leads to a successful destination."

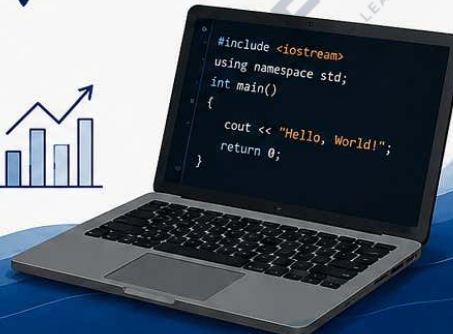
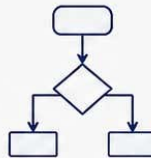
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



"Clear requirements
build the right software."

Software Requirements

Functional and Non-Functional Requirements

"Good software
starts with
understanding
user needs."

1 What are Software Requirements ?

Software requirements are the needs or expectations of users and stakeholders that a software system must fulfill. They describe what the system should do and how well it should do it.



"Requirements are the foundation
of a successful software project."

4 Non-Functional Requirements

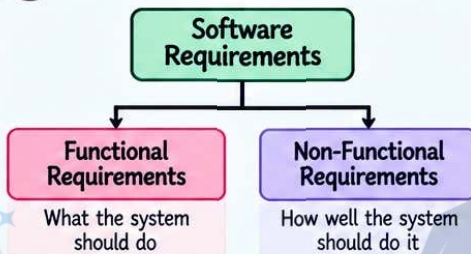
- Describe the quality attributes or constraints of the system.
- Define how the system should perform its functions.
- Concerned with performance, security, usability, reliability, maintainability, etc.
- Do not specify any particular function, but set the criteria for system operation.
- Often more difficult to measure than functional requirements.



Example:

- The system shall respond to user requests within 2 seconds.
- The system shall be available 99.9% of the time.
- The system shall encrypt user data for security.

2 Types of Software Requirements



5 Differences between Functional and Non-Functional Requirements

Basis	Functional Requirements	Non-Functional Requirements
Purpose	Specify what the system should do	Specify how well the system should do it
Focus	System features and behavior	Quality attributes and constraints
Examples	Login, Register, Generate Report	Performance, Security, Usability
Nature	Business/User needs	Technical and quality needs
Measurable	Easily measurable	Often difficult to measure
Priority	Mandatory for system operation	Important for system effectiveness

3 Functional Requirements

- Describe the specific functions or features that the system must perform.
- Define the behavior of the system.
- Based on user needs and business requirements.
- Usually expressed as inputs, processes and outputs.
- Can be mandatory or optional.



Example:

- The system shall allow users to register an account.
- The system shall authenticate the user using username and password.
- The system shall generate a report in PDF format.

6 Importance

- Helps in understanding user needs clearly.
- Acts as a basis for design, development and testing.
- Reduces the risk of project failure.
- Improves communication among stakeholders.
- Helps in estimating cost, time and resources.
- Ensures the development of a high-quality system.



"Well-defined requirements save time, cost and effort."

7 Key Takeaways

- Functional requirements define WHAT the system should do.
- Non-functional requirements define HOW WELL it should do.
- Both types are essential for a successful software project.
- Clear and complete requirements lead to better design, development and user satisfaction.
- Requirements form the base for all other SDLC activities.

Right Requirements → Better Development → Successful Software

"Understand Today,
Build a Better Tomorrow."

"Understand the user,
translate the need,
build the right system."

Software Requirements

User Requirements and System Requirements

"From user needs
to system design
— that's the path
to successful software."

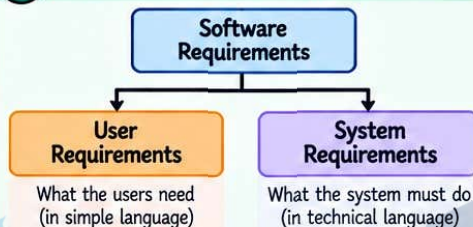
1 What are Software Requirements ?

Software requirements are the needs or expectations of users and other stakeholders that a software system must fulfill. They describe what the system should do and how well it should do it.



"Requirements are the foundation of a successful software project."

2 Types of Software Requirements



3 User Requirements

- Also called end-user requirements.
- Describe the needs of the users in simple, natural language.
- Focus on what the user expects from the system.
- Written from the user's perspective (non-technical).
- Used to communicate with clients, customers and stakeholders.
- Basis for defining system requirements.



Users

"User requirements tell us **WHAT** the user needs."

4 System Requirements

- Also called technical requirements.
- Describe the functions, features, constraints and behaviors of the system in technical terms.
- Focus on how the system will meet the user needs.
- Written for developers, designers and testers.
- Include functional and non-functional requirements and detailed specifications.
- Used as a basis for design, development, testing and implementation.



"System requirements tell us **HOW** the system will work to meet the user needs."

5 Comparison: User vs System Requirements

Basis	User Requirements	System Requirements
Purpose	Capture user needs and expectations	Define detailed technical specifications
Language	Simple, non-technical (natural language)	Technical, precise (technical language)
Focus	What the user wants	How the system will work
Written by	Users, clients, stakeholders	System analysts, developers
Audience	Non-technical people	Technical team
Level of Detail	High-level and brief	Detailed and specific
Example	"User can log in"	"System shall authenticate user using username and password with encryption (AES)."

6 Examples

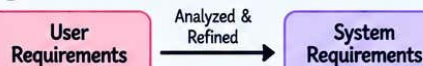
User Requirements (in simple language)

- User should be able to register an account.
- User should be able to log in securely.
- User should be able to search for products.
- User should receive notifications for new updates.
- User should be able to view order history.

System Requirements (in technical language)

- The system shall provide a registration module with email verification.
- The system shall authenticate users using username and password with encrypted storage.
- The system shall allow product search using keywords with a response time < 2 seconds.
- The system shall send notifications via email/SMS using a third-party service.
- The system shall store order history in a database and allow retrieval within 3 seconds.

7 Relationship



User requirements are the starting point, and system requirements are the detailed technical form derived from them.

8 Key Takeaways

- ✓ User requirements define **WHAT** the user needs.
- ✓ System requirements define **HOW** the system will meet those needs.
- ✓ User requirements are simple and non-technical.
- ✓ System requirements are detailed and technical.
- ✓ Both are essential for successful software development.
- ✓ Clear requirements reduce risks, save time and improve quality.

9 Conclusion

Understanding user requirements and translating them into system requirements is a critical step in software engineering. Clear, complete and consistent requirements ensure the development of a high-quality system that meets user needs and business goals.



"Happy Users + Well-Defined System = Successful Software"

"Good requirements lead to great software."

Requirements Engineering Process

Requirements Elicitation and Analysis

Understand
→ Analyze
→ Build the Right Solution.



Requirements engineering is the process of discovering, understanding, analyzing, documenting and managing the needs of stakeholders for a software system.

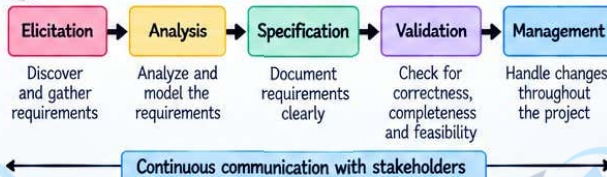
1 What is Requirements Engineering ?

Requirements engineering (RE) is a systematic process of identifying, eliciting, analyzing, documenting, validating and managing the requirements of a software system.



"It ensures that the right product is built for the right people."

2 Requirements Engineering Process



3 What is Requirements Elicitation ?

Requirements elicitation is the process of discovering and gathering the needs, expectations and constraints of stakeholders for the proposed system.

"It answers the question 'What do the users need?'"



4 Goals of Requirements Elicitation

- ✓ Understand the problem domain
- ✓ Identify all possible requirements
- ✓ Capture needs from all stakeholders
- ✓ Discover constraints and dependencies
- ✓ Avoid missing or ambiguous requirements
- ✓ Build a common understanding



The goal is to capture the real needs, not just the stated needs.

5 Techniques for Requirements Elicitation



6 What is Requirements Analysis ?

Requirements analysis is the process of examining, organizing, refining and modeling the gathered requirements to understand their feasibility, consistency and completeness.

"It answers the question 'What do the requirements mean and how can they be achieved?'"



7 Activities in Requirements Analysis

- ✓ Classify and organize requirements
- ✓ Remove ambiguities and inconsistencies
- ✓ Identify and resolve conflicts
- ✓ Prioritize requirements
- ✓ Model requirements (e.g., use cases, data flow, etc.)
- ✓ Assess feasibility (technical, economic, schedule)
- ✓ Define acceptance criteria



8 Outputs of Elicitation and Analysis

- ✓ List of functional requirements
- ✓ List of non-functional requirements
- ✓ Use cases / user stories
- ✓ Requirement models (UML, DFD, etc.)
- ✓ Prioritized and validated requirements
- ✓ Feasibility report
- ✓ Basis for requirements specification



Structured and clear requirements

9 Key Challenges

- 1 Incomplete or unclear requirements
- 1 Conflicting stakeholder needs
- 1 Changing requirements
- 1 Hidden requirements
- 1 Communication gaps
- 1 Unrealistic expectations
- 1 Difficulty in prioritization

10 Key Takeaways

- ✓ Requirements elicitation finds the needs.
- ✓ Requirements analysis understands and refines them.
- ✓ Both are critical for building the right software.
- ✓ Use a combination of techniques.
- ✓ Involve all stakeholders.
- ✓ Clear and well-analyzed requirements reduce risks, cost and rework.



Better Requirements → Better Design → Better Software

"Right Requirements Today,
A Successful Tomorrow."

**"Right Requirements
Reduce Risks,
Save Time,
Build Better Software."**

Requirements Engineering Process

Requirements Validation and Requirements Management

**Manage
Requirements
Today for a
Better Tomorrow.**

1 Recap: Requirements Engineering Process

Requirements engineering is a systematic process of identifying, eliciting, analyzing, documenting, validating and managing the requirements of a software system.



2 What is Requirements Validation ?

Requirements validation is the process of checking whether the documented requirements are correct, complete, consistent, feasible and meet the needs of the stakeholders.



**"Validation answers the question
– Are we building the right product?"**

3 Objectives of Requirements Validation

- ✓ Ensure requirements are correct and error-free.
- ✓ Check completeness (no important requirement is missing).
- ✓ Ensure consistency (no conflicts among requirements).
- ✓ Verify feasibility (technical, functional and economic).
- ✓ Confirm that requirements meet stakeholder needs.
- ✓ Increase the chances of customer satisfaction.
- ✓ Reduce risks of rework and cost overruns.
- ✓ Establish a baseline for system development.

4 Techniques for Requirements Validation

- | | |
|------------------------|--|
| 1 Reviews | Formal or informal review of requirements by stakeholders, developers and experts. |
| 2 Walkthroughs | Step-by-step examination of requirements with the project team. |
| 3 Inspections | Detailed and structured examination to find defects or inconsistencies. |
| 4 Prototyping | Build prototypes to validate requirements with users. |
| 5 Test Case Derivation | Create test cases from requirements to check their correctness and completeness. |
| 6 Modeling (e.g., UML) | Use diagrams and models to validate requirements visually. |
| 7 User Acceptance | Get approval from end-users and customers. |

5 What is Requirements Management ?

Requirements management is the process of tracking, controlling and managing changes to requirements throughout the software development lifecycle.



"Requirements management ensures that changing needs are handled in a controlled and systematic way."

6 Activities in Requirements Management

- | | |
|-----------------------|---|
| 1 Tracking | Keep a record of all requirements and their status. |
| 2 Change Control | Evaluate, approve or reject proposed changes. |
| 3 Version Management | Maintain different versions of requirements. |
| 4 Traceability | Maintain links between requirements and design, code, and test cases. |
| 5 Prioritization | Manage and prioritize requirements based on business value. |
| 6 Status Reporting | Provide regular updates to stakeholders. |
| 7 Baseline Management | Establish and maintain a stable set of requirements. |

7 Tools for Requirements Management

- ✓ IBM DOORS
- ✓ Jira
- ✓ Azure DevOps
- ✓ Helix ALM
- ✓ ReqView
- ✓ Rational RequisitePro
- ✓ Version control systems (e.g., Git)



**Tools help
manage
requirements
efficiently.**

8 Benefits of Validation and Management

- | Validation Benefits | Management Benefits |
|--|--|
| ✓ Improves requirement quality | ✓ Better control over changes |
| ✓ Reduces risks and rework | ✓ Improved traceability |
| ✓ Ensures customer satisfaction | ✓ Enhanced communication |
| ✓ Helps in early detection of problems | ✓ Reduced cost and time |
| ✓ Provides a clear understanding to all stakeholders | ✓ Supports project planning |
| | ✓ Maintains consistency throughout the lifecycle |

9 Key Takeaways

- ✓ Requirements validation ensures we build the right product.
- ✓ Requirements management ensures we manage changes effectively.
- ✓ Both are essential parts of the requirements engineering process.
- ✓ Use appropriate techniques and tools for better results.
- ✓ Involving stakeholders throughout leads to high-quality software.



**"Validate the requirements, manage the changes,
and build software that truly delivers value."**

Well-Managed Requirements → Successful Software

**Requirements Today
= Better Solutions Tomorrow.**

"Measure to Manage, Manage to Improve."

Metrics for Process and Products

Software Measurement

"What gets measured, gets improved."



Software measurement is the process of assigning numbers or symbols to the attributes of software products, processes and resources in a systematic way, to improve decision making, control and quality.

1 What is Software Measurement ?

- Software measurement is the process of quantifying the attributes of software process, product and resources.
- It provides objective data for decision making, control and improvement.
- It uses metrics to measure size, effort, quality, productivity, etc.
- Measurement helps in understanding, comparing and predicting software development results.



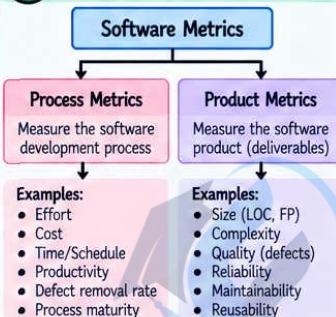
"Measurement brings objectivity to software engineering."

2 Why Measure Software ?

- ✓ Helps in project planning and estimation.
- ✓ Tracks progress and productivity.
- ✓ Ensures product quality.
- ✓ Identifies problems early.
- ✓ Supports process improvement.
- ✓ Enables comparison across projects.
- ✓ Provides a basis for measurement-driven decision making.
- ✓ Reduces risks and uncertainties.

"If you can measure it, you can improve it."

3 Types of Metrics



4 Process Metrics (Examples)

Metric	Description
Effort	Total person-hours spent on the project.
Cost	Total monetary cost.
Schedule/Time	Planned vs actual time.
Productivity	Output per unit effort (e.g., LOC per person-month).
Defect removal rate	Number of defects found and removed during the process.
Process maturity	Level of process capability (e.g., CMMI levels).
Staff turnover	Rate at which team members leave.
Resource utilization	Efficient use of human and technical resources.

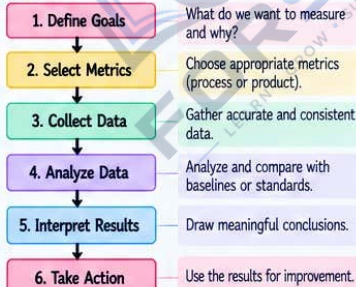
5 Product Metrics (Examples)

Metric	Description
Size	Lines of Code (LOC), Function Points (FP)
Complexity	Cyclomatic complexity, Halstead metrics
Quality	Number of defects, defect density
Reliability	Mean Time Between Failures (MTBF)
Maintainability	Effort required to modify the software
Reusability	Extent to which components can be reused
Testability	Ease of testing the software
Performance	Response time, throughput, memory usage

6 Common Software Size Metrics

Metric	Description
Lines of Code (LOC)	Total number of lines in the source code.
Function Points (FP)	Measures functionality from user perspective (language independent).
Use Case Points (UCP)	Based on use cases and actors.
Object Points	Used in object-oriented systems.
Story Points	Used in Agile projects (relative size).
Kilo Source Lines of Code (KSLOC)	LOC measured in thousands.

7 Measurement Process



8 Key Takeaways

- ✓ Metrics provide objective and quantitative data.
- ✓ Both process and product metrics are important.
- ✓ Measurement helps in control, prediction and improvement.
- ✓ Use appropriate metrics for the right purpose.
- ✓ Regular measurement leads to better quality software.
- ✓ Metrics are not a goal, but a tool for better decision making.

"Measure - Analyze - Improve - Repeat."

9 Conclusion

Software measurement using proper metrics for process and products helps in building high-quality software, improving productivity and ensuring project success. It provides the data needed to make informed decisions and continuously improve the software development process.



"Better Measurement Leads to Better Software."

"Quality is not an act,
it is a measurable habit."

Metrics for Software Quality

Measure Quality → Improve Continuously → Deliver Better Software

"What gets measured,
gets improved."

1 What is Software Quality ?

Software quality means the degree to which a software product satisfies stated and implied requirements and meets user expectations.



"Quality is fitness for use."
– Juran

4 Types of Quality Metrics

Product Metrics

Measure attributes of the software product (e.g., defects, complexity, size).



Process Metrics

Measure the software development process (e.g., review effectiveness, test coverage).



Project Metrics

Measure project-related aspects (e.g., cost, effort, schedule, productivity).



Quality-in-Use Metrics

Measure the quality experienced by end users in the real environment.



7 Benefits of Quality Metrics

- ✓ Objective measurement of quality.
- ✓ Early detection of defects.
- ✓ Better decision making.
- ✓ Improved process and product quality.
- ✓ Higher customer satisfaction.
- ✓ Reduced cost and development time.
- ✓ Helps in continuous improvement.



2 Why Measure Software Quality ?

- ✓ Helps to know the actual quality level.
- ✓ Provides objective and quantitative data.
- ✓ Identifies defects and areas for improvement.
- ✓ Supports quality control and assurance.
- ✓ Improves customer satisfaction.
- ✓ Reduces cost of rework and maintenance.
- ✓ Helps in comparing different releases or products.
- ✓ Aids in meeting industry standards (e.g., ISO, CMMI).

5 Important Metrics for Software Quality

Quality Attribute	Metric	Description
Functionality	Defect density	Number of defects per KLOC (thousand lines of code).
	Requirement coverage	Percentage of requirements implemented tested.
Reliability	Mean Time Between Failures (MTBF)	Average time between failures.
	Failure rate	Number of failures per unit of time.
Usability	User satisfaction score	Measured through user surveys.
	Average learning time	Time taken by a new user to learn the system.
Maintainability	Maintainability Index	Combined measure based on Halstead metrics, LOC, etc.
	Mean Time to Repair (MTTR)	Average time to fix a defect.
Efficiency	Response time	Time taken to respond to a user request.
	Resource utilization	CPU, memory, and storage usage.
Portability	Porting effort	Effort required to move the software to another environment.
	Platform coverage	Number of platforms supported.
Overall Quality	Defect removal efficiency	Percentage of defects removed before release.
	Customer satisfaction	Feedback score from users/customers.

3 Quality Models and Attributes

Common quality models such as ISO 9126 / ISO 25010 define key quality characteristics:



6 Example Values

Metric	Example Target / Value
Defect density	< 1 defect per KLOC
Test coverage	> 90%
MTBF	> 1000 hours
MTTR	< 2 hours
Response time	< 2 seconds
CPU utilization	< 70%
User satisfaction	> 80%
Maintainability Index	> 70 (out of 100)

8 Key Takeaways

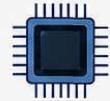
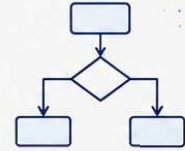
- ✓ Metrics make software quality measurable.
- ✓ Use a mix of product, process, project and user metrics.
- ✓ Choose metrics relevant to your project and goals.
- ✓ Regular measurement and analysis lead to better quality.
- ✓ Quality metrics support continuous improvement.

"Measure today for a better,
more reliable tomorrow."

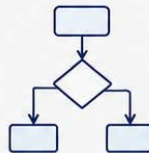


Quality is Measurable → Measure → Analyze → Improve → Deliver Value

1010101010
0101010101
1010101010
0101010101



10110
01001
10101



INTRODUCTION TO JAVA : HISTORY AND EVOLUTION

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems. It is designed to have as few implementation dependencies as possible.

"Write Once, Run Anywhere"

1. HISTORY OF JAVA

- Java was developed by James Gosling and his team at Sun Microsystems (now a subsidiary of Oracle).
- The project was started in 1991 under the name "Oak".
- Initially designed for interactive television, set-top boxes and other consumer electronic devices.
- In 1995, Oak was renamed "Java", inspired by Java coffee, which is known for its richness and energy.
- Sun Microsystems officially released Java in 1996.
- In 2009, Oracle Corporation acquired Sun Microsystems and now manages and maintains Java.

2. KEY FEATURES OF JAVA (AT INTRODUCTION)

- | | |
|--|--------------------|
| ★ Simple | ★ Portable |
| ★ Object-Oriented | ★ High Performance |
| ★ Platform Independent
(Write Once, Run Anywhere) | ★ Multithreaded |
| ★ Secure | ★ Distributed |
| ★ Robust | ★ Interpreted |
| ★ Architecture Neutral | |



NOTE :

Java was designed to be reliable, secure and portable so that it can be used to develop applications for a wide range of platforms and devices.

3. EVOLUTION OF JAVA

- | | | |
|------|---|--|
| 1991 | → | The Green Project
James Gosling and his team started the project. |
| 1992 | → | Project Oak
The language was initially called Oak. |
| 1995 | → | Renamed as Java
Oak was renamed Java. |
| 1996 | → | JDK 1.0
Official release of Java (JDK 1.0) by Sun Microsystems. |
| 1997 | → | JDK 1.1
Added features like Inner Classes, Event Handling, JDBC, RMI, etc. |
| 1998 | → | J2SE 1.2
Performance improvements and important APIs |
| 2000 | → | J2SE 1.3
HotSpot JVM, Java Sound, RMI improvements, etc. |
| 2004 | → | J2SE 5.0 (Java 5)
Generics, Annotations, For-each loop, Autoboxing, Enums, Enhanced library. |
| 2006 | → | Java SE 6
Scripting support, Performance enhancements, etc. |
| 2011 | → | Java SE 7
Diamond operator, Try-with-resources, NIO.2, etc. |
| 2014 | → | Java SE 8
Lambda expressions, Stream API, Date/Time API, etc. |
| 2017 | → | Java SE 9
Module system (Project Jigsaw), JShell, etc. |
| 2021 | → | Java SE 17 (LTS)
Long Term Support release with performance and security improvements. |

KEY POINTS

- ✓ Java has come a long way from a small project "Oak" to a powerful, widely-used programming language.
- ✓ It has evolved continuously with new features and improvements.
- ✓ Java is now used in web, mobile, enterprise applications, Android, cloud, big data, AI/ML and more.

JAVA : FEATURES

Java is a powerful, general-purpose, object-oriented programming language. It provides many features that make it simple, secure, platform-independent and widely used.

KEY POINTS

- ★ Java is simple, object-oriented and secure.
- ★ It is platform independent and portable.
- ★ It provides high performance and multithreading.
- ★ It is robust, distributed and has dynamic nature.
- ★ Java supports automatic memory management.

MAJOR FEATURES OF JAVA

1 Simple



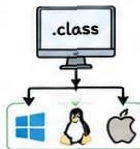
- Java has a simple syntax similar to C/C++.
- It removes complex features like pointers, operator overloading, etc.

2 Object-Oriented



- Java is based on OOPs concepts.
- It supports classes, objects, inheritance, polymorphism, encapsulation and abstraction.

3 Platform Independent



- Java code is compiled to bytecode (.class file)
- Bytecode runs on any platform using JVM.
- Write Once, Run Anywhere.

4 Portable



- Java supports different platforms (Windows, Linux, macOS, etc.).
- It has fixed size data types, which makes it portable.

5 Secure



- Java provides built-in security features.
- No explicit pointer usage, no access to memory addresses.
- It supports authentication and encryption.

6 Robust



- Strong memory management with garbage collection.
- Exception handling and type checking make Java robust.

7 High Performance



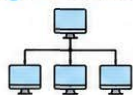
- Java uses Just-In-Time (JIT) compiler.
- Converts bytecode to native code at runtime for better performance.

8 Multithreaded



- Java supports multithreading.
- Multiple threads can run concurrently.
- Improves performance of applications.

9 Distributed



- Java is designed for distributed computing.
- It supports APIs like RMI, Socket programming, etc.

10 Architecture Neutral



- Java compiler generates architecture-neutral bytecode.
- Bytecode runs on any processor with JVM.

11 Interpreted



- Java bytecode is interpreted by JVM line by line.
- Easy to debug and test.

12 Dynamic



- Java is dynamic.
- It can adapt to an evolving environment.
- Classes are loaded at runtime.

13 Automatic Memory Management



- Java has Garbage Collection.
- It automatically allocates and de-allocates memory.
- No manual memory management.

14 Strongly Typed



- Java is strongly typed.
- All variables must be declared before use.
- Ensures type safety.

15 High Reliability



- Java provides reliable features like exception handling, garbage collection, etc.
- Reduces system failure.

16 Rich API



- Java provides a rich set of libraries (APIs).
- Helps in rapid development.
- Example: Collections, I/O, Networking, etc.

NOTE



These features make Java a powerful, flexible, secure and widely used programming language for developing desktop, web, mobile and enterprise applications.

In Short

Simple • Object-Oriented • Platform Independent • Portable • Secure • Robust
High Performance • Multithreaded • Distributed • Dynamic • Rich API and more.

JRE (Java Runtime Environment)

JRE (Java Runtime Environment) is a part of Java Software that provides **libraries, Java Virtual Machine (JVM)** and other components required to **run** Java programs.

KEY POINTS

- ✓ JRE is required to **run** Java applications.
- ✓ It does not contain tools for development.
- ✓ It includes JVM, class libraries and other supporting files.
- ✓ It is platform dependent.
- ✓ It is a part of JDK (Java Development Kit).

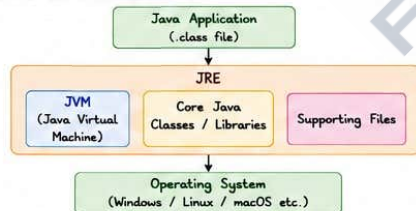
1. WHAT IS JRE?

- JRE provides the environment in which Java bytecode can be executed.
- It includes JVM, core class libraries and other supporting files.
- It is used to **run** Java applications on any platform.

2. COMPONENTS OF JRE

- **JVM (Java Virtual Machine)**
Executes the Java bytecode.
- **Core Java Classes / Libraries**
A set of standard class libraries required by Java programs.
- **Supporting Files**
Resource files and configurations required by JRE.

3. JRE ARCHITECTURE



7. EXAMPLE

To run a Java program:

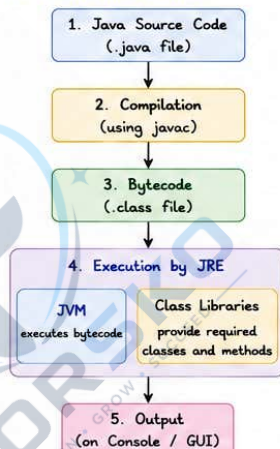
1. Write Program → Hello.java
2. Compile → javac Hello.java
3. Run → java Hello

- When the command "java Hello" is executed, JRE is used to run the program.

Output:

Hello, Welcome to Java!

4. HOW JRE WORKS?



1. Java Source Code

Programmer writes code in .java file.

2. Compilation

The source code is compiled using javac compiler to generate bytecode (.class file).

3. Bytecode

Bytecode is platform independent code generated after compilation.

4. Execution by JRE

- JVM loads and verifies the bytecode.
- JVM executes the bytecode.
- Class libraries provide the required classes and methods to the program.

5. Output

The result is displayed on the console or in the GUI.

5. JRE vs JDK

Feature	JRE	JDK
Full Form	Java Runtime Environment	Java Development Kit
Purpose	To run Java programs	To develop and run Java programs
Contains	JVM, Class Libraries, Supporting Files	JRE + Development Tools (javac, java, jdb, javadoc, etc.)
Used By	End Users	Developers
Installation Size	Smaller	Larger
Example	To run .class file	To compile and run .java file

6. BENEFITS OF JRE

- ★ Enables platform independence.
- ★ Ensures reliability and security.
- ★ Automatic memory management through Garbage Collection.
- ★ Provides rich set of libraries.
- ★ Makes Java programs portable and easy to run.

8. JRE FOLDER STRUCTURE (Example)

```
jre/
├── bin/      → Contains executables (java, etc.)
├── lib/      → Core Libraries (rt.jar, etc.)
├── conf/     → Configuration files
├── legal/    → License and legal information
└── other supporting files and folders
```

9. IMPORTANT NOTE



- JRE is required only to run compiled Java programs.
- JRE does not provide any tool to write or compile programs.
- For development and execution, JDK is required.

JDK (Java Development Kit) and JVM (Java Virtual Machine)

JDK is a software development kit used to develop, compile, debug and run Java programs.

JVM is the runtime environment that executes Java bytecode and provides platform independence.

KEY POINTS

- ✓ JDK is used for development.
- ✓ JVM is used for execution.
- ✓ JDK includes JRE and development tools.
- ✓ JVM executes bytecode generated by the compiler.
- ✓ JVM makes Java platform independent.

1. JDK (Java Development Kit)

1.1 What is JDK?

- JDK is a complete development environment for Java.
- It is used by developers to create, build, test and run Java applications.
- It includes tools, libraries, JRE and JVM.

1.2 Components of JDK

- ➔ **1. JRE (Java Runtime Environment)**
Provides libraries, JVM and other files required to run Java programs.
- ➔ **2. JVM (Java Virtual Machine)**
Executes Java bytecode.
- ➔ **3. Development Tools**
Tools used to develop and debug Java programs.
- ➔ **4. Libraries**
Collection of useful classes and interfaces.

1.3 Structure of JDK



1.4 Tools Provided by JDK

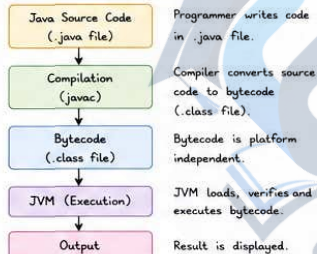
- **javac** - Java compiler (compiles .java to .class)
- **java** - Runs the Java program
- **jdb** - Java debugger
- **javadoc** - Generates documentation
- **jar** - Creates Java archive (.jar) files
- **javap** - Disassembles class files
- **jconsole** - Monitoring and management tool
- **jdeps** - Analyzes dependencies

2. JVM (Java Virtual Machine)

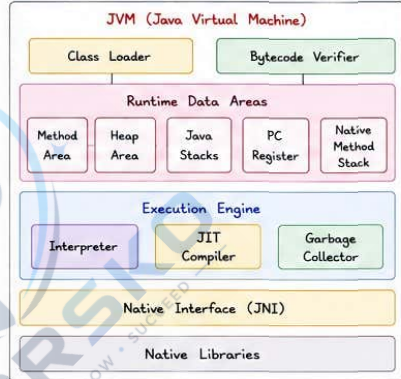
2.1 What is JVM?

- JVM is a runtime engine that executes Java bytecode.
- It provides platform independence.
- It is a part of JRE.
- One JVM can run on many operating systems.

2.2 How JVM Works?



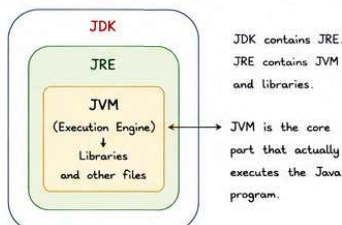
2.3 Architecture of JVM



3. JDK vs JVM

Feature	JDK	JVM
Full Form	Java Development Kit	Java Virtual Machine
Purpose	Used for development of Java applications	Used for execution of Java applications
Contains	JRE, JVM, tools (javac, java, jdb, etc.), libraries	Class loader, verifier, runtime data areas, execution engine, etc.
Used By	Developers	End users
Input	.java source file	.class bytecode
Output	.class file (after compilation) and also runs program	Executes bytecode and produces output
Dependency	Platform dependent (installation)	Platform independent (bytecode level)

4. Relationship



5. Example

Program: Hello.java

```
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

Steps:

1. Write program in Hello.java
2. Compile using: `javac Hello.java` → Hello.class
3. Run using: `java Hello`
4. JVM executes Hello.class and displays output.

Output:

Hello, Java!



Important Note

- Always use JDK for development.
- Install JDK to get both development tools and runtime environment.

- JVM is available on different platforms, so the same .class file can run anywhere.
- "Write Once, Run Anywhere" is possible because of JVM.

DATA TYPES IN JAVA

- ✓ In Java, data types specify the type of data a variable can hold.
- ✓ Java data types are divided into two categories:
 1. **Primitive Data Types**
 2. **Non-Primitive (Reference) Data Types**

1. PRIMITIVE DATA TYPES (Built-in / Basic)

Data Type	Size	Default Value	Description	Example
byte	1 byte	0	Stores whole numbers from -128 to 127	<code>byte b = 10;</code>
short	2 bytes	0	Stores whole numbers from -32,768 to 32,767	<code>short s = 200;</code>
int	4 bytes	0	Stores whole numbers (larger range)	<code>int i = 1000;</code>
long	8 bytes	0L	Stores very large whole numbers	<code>long l = 123456789L;</code>
float	4 bytes	0.0f	Single precision floating point number	<code>float f = 5.6f;</code>
double	8 bytes	0.0d	Double precision floating point number	<code>double d = 3.14159;</code>
char	2 bytes	'\u0000'	Stores a single character	<code>char ch = 'A';</code>
boolean	1 bit	false	Stores true or false values	<code>boolean b = true;</code>

2. NON-PRIMITIVE (REFERENCE) DATA TYPES

Type	Description
String	Represents a sequence of characters.
Array	Stores multiple values of same type.
Class	User-defined data type (Objects).
Interface	Collection of abstract methods.

- ★ IMPORTANT**
- ✓ Primitive types store actual values.
 - ✓ Non-primitive types store references (addresses).



NOTE



Java is a strongly typed language, so every variable must have a data type.

KEYWORDS IN JAVA

- ★ **Keywords** are reserved words in Java that have special meaning. They cannot be used as variable names, method names or class names.

CATEGORIES OF KEYWORDS

1. Access Modifiers

- public
- private
- protected
- default (no keyword)

2. Class & Object Related

- class
- extends
- interface
- this
- object
- implements
- new
- super

3. Control Flow

- if
- else
- switch
- case
- default
- for
- while
- do
- break
- continue
- return

4. Data Types

- boolean
- short
- long
- double
- byte
- int
- float
- char

5. Exception Handling

- try
- finally
- throws
- catch
- throw

6. Other Keywords

- static
- abstract
- synchronized
- transient
- final
- native
- volatile
- strictfp

COMMONLY USED JAVA KEYWORDS (Alphabetical Order)

abstract	assert	boolean	break	byte	case	catch	char
class	const	continue	default	do	double	else	enum
extends	final	finally	float	for	goto*	if	implements
import	instanceof	int	interface	long	native	new	package
private	protected	public	return	short	static	strictfp	super
switch	synchronized	this	throw	throws	transient	try	void
volatile	while	true	false	null			

EXAMPLES

- ✓ Valid variable name : `int age;`
- ✗ Invalid variable name : `int class;` // class is keyword
- ✓ Valid method name : `void showData() {}`
- ✗ Invalid method name : `void if() {}` // if is keyword



TIPS

- ✓ Keywords are all lowercase.
- ✓ true, false and null are also special keywords.

JAVA OPERATORS

Operators are special symbols that are used to perform operations on variables and values.

Java is rich in built-in operators.

1. TYPES OF OPERATORS IN JAVA

- Arithmetic Operators
- Relational (Comparison) Operators
- Logical Operators
- Bitwise Operators
- Assignment Operators
- Unary Operators
- Ternary (Conditional) Operator
- Type Comparison Operator

2. ARITHMETIC OPERATORS

Operator	Meaning	Example	Result
+	Addition	int a = 10 + 5;	15
-	Subtraction	int a = 10 - 5;	5
*	Multiplication	int a = 10 * 5;	50
/	Division	int a = 10 / 5;	2
%	Modulus (Remainder)	int a = 10 % 3;	1

Note: If either operand is double, the result is double.

3. RELATIONAL (COMPARISON) OPERATORS

Operator	Meaning	Example	Result
==	Equal to	10 == 10	true
!=	Not equal to	10 != 20	true
>	Greater than	10 > 5	true
<	Less than	10 < 5	false
>=	Greater than or equal to	10 >= 10	true
<=	Less than or equal to	10 <= 5	false

4. LOGICAL OPERATORS

Operator	Meaning	Example	Result
&&	Logical AND	(10 > 5) && (20 > 10)	true
	Logical OR	(10 > 5) (20 < 10)	true
!	Logical NOT	!(10 > 5)	false

Note: Logical operators work on boolean values (true/false).

5. BITWISE OPERATORS

Operator	Meaning	Example	Result
&	Bitwise AND	5 & 3 (0101 & 0011)	1 (0001)
	Bitwise OR	5 3 (0101 0011)	7 (0111)
^	Bitwise XOR	5 ^ 3 (0101 ^ 0011)	6 (0110)
~	Bitwise Complement	~5 (~0000 0101)	-6
<<	Left Shift	5 << 1	10 (1010)
>>	Right Shift	5 >> 1	2 (0010)
>>>	Unsigned Right Shift	5 >>> 1	2 (0010)

Note: Bitwise operators work on integral types.

6. ASSIGNMENT OPERATORS

Operator	Example	Equivalent To
=	a = 10	a = 10
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5
&=	a &= 5	a = a & 5
=	a = 5	a = a 5
^=	a ^= 5	a = a ^ 5
<<=	a <<= 2	a = a << 2
>>=	a >>= 2	a = a >> 2
>>>=	a >>>= 2	a = a >>> 2

9. OPERATOR PRECEDENCE

Precedence	Operator	Description
1	()	Parentheses
2	++ -- + - ! ~	Unary
3	* / %	Multiplicative
4	+ -	Additive
5	<< >> >>>	Shift
6	< <= > >=	Relational
7	== !=	Equality
8	&	Bitwise AND
9	^	Bitwise XOR
10		Bitwise OR
11	&&	Logical AND
12		Logical OR
13	?:	Ternary
14	= += -= *= /= %= &= = ^= <<= >>= >>>=	Assignment

Note: Higher precedence operator is evaluated first.

7. UNARY OPERATORS

Operator	Meaning	Example	Result
++	Increment (increase by 1)	int a = 5; a++;	a = 6
--	Decrement (decrease by 1)	int a = 5; a--;	a = 4
+	Unary Plus (no change)	int a = 5; +a;	a = 5
-	Unary Minus (change sign)	int a = 5; -a;	a = -5
!	Logical NOT (invert)	boolean b = true; !b;	false
~	Bitwise Complement	int a = 5; ~a;	-6

8. TERNARY (CONDITIONAL) OPERATOR

Condition ? expression1 : expression2
If condition is true, expression1 is executed
otherwise expression2 is executed.

Example:

```
int max = (a > b) ? a : b;
```

Equivalent to:

```
int max;  
if(a > b)  
    max = a;  
else  
    max = b;
```

10. TYPE COMPARISON OPERATOR

instanceof

Used to test whether an object is an instance of a class, subclass or interface.

Example:

```
String s = "Hello";  
System.out.println(s instanceof String); // true  
System.out.println(s instanceof Object); // true
```

IMPORTANT POINTS

- Operators are used with operands (variables, values, expressions).
- The result of an operation depends on the type of operands.
- In case of different data types, Java automatically promotes smaller type to larger type.

EXAMPLES

```
int a = 10, b = 3;  
a + b = 13  
a * b = 30  
a > b && a < 20 // true
```

```
int x = 5;  
x++ ; // 6  
++x ; // 7  
x > 5 ? "Yes" : "No" // Yes
```

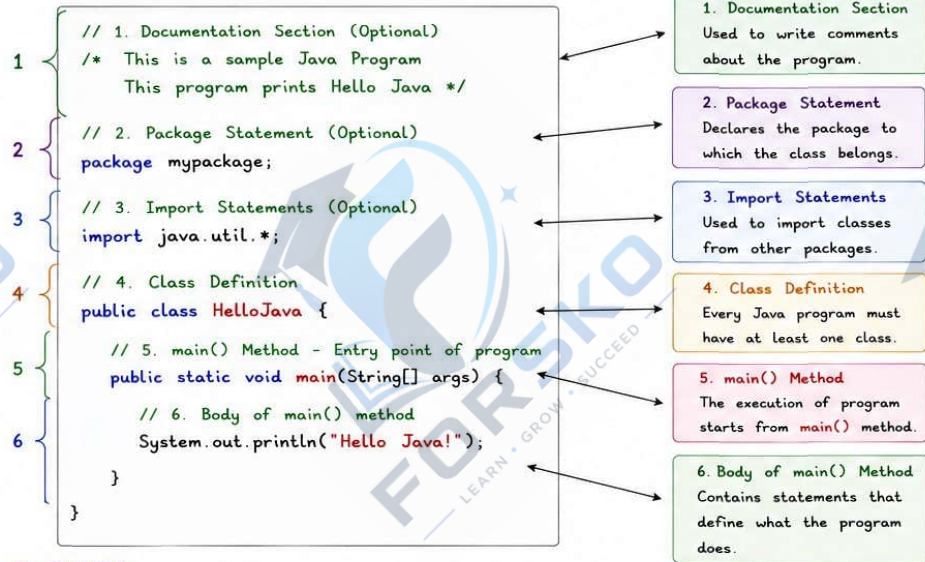
NOTE

Knowing operators and their precedence is very important for writing correct and efficient Java programs.

STRUCTURE OF JAVA PROGRAM

A Java program is a collection of classes. Every Java program must contain a class with **main()** method. The execution of program starts from the **main()** method.

1. GENERAL FORM OF JAVA PROGRAM



4. EXAMPLE

Program

```
package mypackage;
import java.util.*;
public class HelloJava {
    public static void main(String[] args) {
        System.out.println("Hello Java!");
    }
}
```

Output

Hello Java!

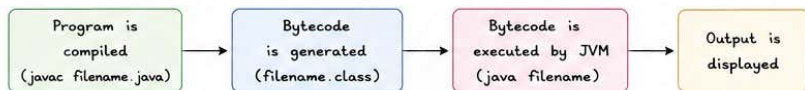
2. EXPLANATION OF PARTS

Part	Description
Documentation Section	Comments that describe the program.
Package Statement	(Optional) Specifies the package name using package keyword.
Import Statements	(Optional) Imports classes/interfaces from other packages using import keyword.
Class Definition	Defines a class using class keyword. The file name should be same as the public class name.
main() Method	public static void main(String[] args) is the entry point of every Java program.
Body of main() Method	Contains the executable statements of the program.

3. POINTS TO REMEMBER

- ✓ Every Java program must have main() method.
- ✓ Class name and file name must be same (if class is public).
- ✓ Java is case-sensitive.
- ✓ Statements are executed in sequence inside main() method.

5. FLOW OF EXECUTION



JAVA COMMAND LINE ARGUMENTS

Command line arguments are the values passed to a Java program at the time of running from the command prompt or terminal. These arguments are received by the `main()` method.

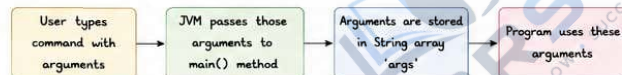
1. What are Command Line Arguments?

- When we run a Java program, we can pass some values along with the program name.
- These values are called command line arguments.
- In Java, command line arguments are received in the `String` array passed to `main()` method.

2. Syntax

```
public class ClassName {  
    public static void main(String[] args) {  
        // args is an array that stores command line arguments  
    }  
}
```

3. How it Works?



4. Example Program

```
public class CommandLineDemo {  
    public static void main(String[] args) {  
        System.out.println("Total arguments: " + args.length);  
        for (int i = 0; i < args.length; i++) {  
            System.out.println("args[" + i + "] = " + args[i]);  
        }  
    }  
}
```

9. Example: Add Two Numbers Using Command Line Arguments

```
public class CommandLineDemo {  
    public static void main(String[] args) {  
        if (args.length != 2) {  
            System.out.println("Please provide two numbers");  
            return;  
        }  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
        int sum = a + b;  
        System.out.println("Sum = " + sum);  
    }  
}
```

5. Running the Program

Compile the program

```
javac CommandLineDemo.java
```

Run the program with arguments

```
java CommandLineDemo 10 Hello Java Programming
```

6. Output

```
Total arguments: 3  
args[0] = 10  
args[1] = Hello  
args[2] = Java  
args[3] = Programming
```



Note

- The first argument is at index 0.
- If no arguments are passed, `args.length` will be 0.
- All arguments are treated as `String` in Java.

7. Accessing Arguments

Expression	Description
<code>args.length</code>	Returns total number of arguments
<code>args[i]</code>	Returns argument at index i
<code>args[0]</code>	First argument
<code>args[args.length - 1]</code>	Last argument

8. Points to Remember

- ✓ `main()` must have `String[] args` to receive command line arguments.
- ✓ Arguments are separated by spaces while running.
- ✓ Order of arguments is preserved.
- ✓ Arguments are always passed as `Strings`.

Run:

```
javac AddTwoNumbers.java
```

```
java AddTwoNumbers 25 37
```

Output:

```
Sum = 62
```

If wrong arguments:

```
java AddTwoNumbers 25
```

Output:

```
Please provide two numbers
```

SIMPLE JAVA PROGRAMS

Here are some simple Java programs for beginners.

1. Print "Hello, World!"

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Output:

Hello, World!

2. Add Two Numbers

```
public class AddTwoNumbers {  
    public static void main(String[] args) {  
        int a = 10, b = 20, sum;  
        sum = a + b;  
        System.out.println("Sum = " + sum);  
    }  
}
```

Output:

Sum = 30

3. Find the Largest Number

```
public class LargestNumber {  
    public static void main(String[] args) {  
        int a = 25, b = 40, c = 15, largest;  
        largest = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);  
        System.out.println("Largest = " + largest);  
    }  
}
```

Output:

Largest = 40

4. Check Even or Odd

```
public class EvenOdd {  
    public static void main(String[] args) {  
        int num = 7;  
        if (num % 2 == 0)  
            System.out.println(num + " is Even");  
        else  
            System.out.println(num + " is Odd");  
    }  
}
```

Output:

7 is Odd

5. Factorial of a Number

```
public class Factorial {  
    public static void main(String[] args) {  
        int n = 5, fact = 1;  
        for (int i = 1; i <= n; i++)  
            fact = fact * i;  
        System.out.println("Factorial of " + n + " = " + fact);  
    }  
}
```

Output:

Factorial of 5 = 120

6. Table of a Number

```
public class Table {  
    public static void main(String[] args) {  
        int n = 5;  
        for (int i = 1; i <= 10; i++)  
            System.out.println(n + " x " + i + " = " + (n * i));  
    }  
}
```

Output:

5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
...
5 x 10 = 50

7. Sum of First N Natural Numbers

```
public class SumNatural {  
    public static void main(String[] args) {  
        int n = 10, sum = 0;  
        for (int i = 1; i <= n; i++)  
            sum = sum + i;  
        System.out.println("Sum = " + sum);  
    }  
}
```

Output:

Sum = 55

8. Reverse a Number

```
public class ReverseNumber {  
    public static void main(String[] args) {  
        int num = 12345, rev = 0, rem;  
        while (num != 0) {  
            rem = num % 10;  
            rev = rev * 10 + rem;  
            num = num / 10;  
        }  
        System.out.println("Reverse = " + rev);  
    }  
}
```

Output:

Reverse = 54321

9. Check Prime Number

```
public class PrimeNumber {  
    public static void main(String[] args) {  
        int num = 17, i, flag = 0;  
        if (num <= 1)  
            flag = 1;  
        else {  
            for (i = 2; i <= num/2; i++) {  
                if (num % i == 0) { flag = 1; break; }  
            }  
        }  
        if (flag == 0)  
            System.out.println(num + " is Prime");  
        else  
            System.out.println(num + " is Not Prime");  
    }  
}
```

Output:

17 is Prime

10. Simple Interest

```
public class SimpleInterest {  
    public static void main(String[] args) {  
        double p = 1000, r = 5.5, t = 2, si;  
        si = (p * r * t) / 100;  
        System.out.println("Simple Interest = " + si);  
    }  
}
```

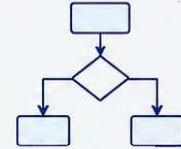
Output:

Simple Interest = 110.0

Note:

- Java is case-sensitive.
- Class name should be same as file name.
- Use meaningful variable names.

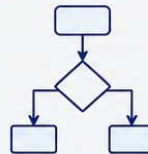
1010101010
0101010101
1010101010
0101010101



Unit - IV



10110
01001
10101



Decision Making and Branching: Simple if statement

1. Introduction



The **if** statement is used to execute a block of code only when a specified condition is true.

★ It is the simplest form of decision making in Java.

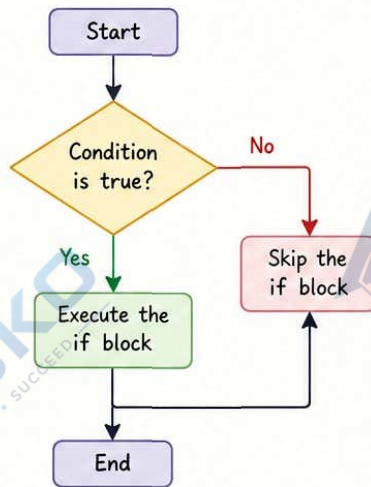
3. Syntax

```
if (condition) {  
    // statements to be executed  
}
```

Example Code

```
int age = 18;  
if (age >= 18) {  
    System.out.println("You are eligible to vote.");  
}
```

2. Flowchart



4. Example

```
public class IfExample {  
    public static void main(String[] args) {  
        int number = 10;  
        if (number > 0) {  
            System.out.println("Number is positive");  
        }  
    }  
}
```

Output:
Number is positive

5. Key Points

- If the condition is **true**, the block inside **if** is executed.
- If the condition is **false**, the block is skipped.
- **If** has no **else** part.

6. Notes

- The condition must be a boolean expression.
- Use simple **if** when you want to execute a block only in one case.



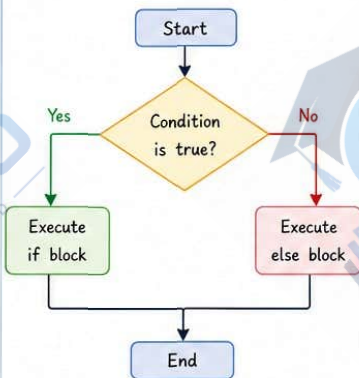
★ Simple if statement is the building block for more complex decision making like **if-else**, **else-if ladder** and **nested if**.

Decision Making and Branching in Java

1. if...else statement

The **if...else** statement is used to execute one block of code if a condition is **true** and another block if the condition is **false**.

Flowchart:



Output:

Number is negative

Key Points:

- If the condition is **true**, the **if** block is executed.
- If the condition is **false**, the **else** block is executed.
- Only one block (**if** or **else**) is executed.

Syntax:

```
if (condition) {  
    // statements when condition is true  
}  
else {  
    // statements when condition is false  
}
```

Example:

```
public class IfElseExample {  
    public static void main(String[] args) {  
        int num = -5;  
        if (num >= 0) {  
            System.out.println("Number is positive");  
        }  
        else {  
            System.out.println("Number is negative");  
        }  
    }  
}
```

Example (Another):

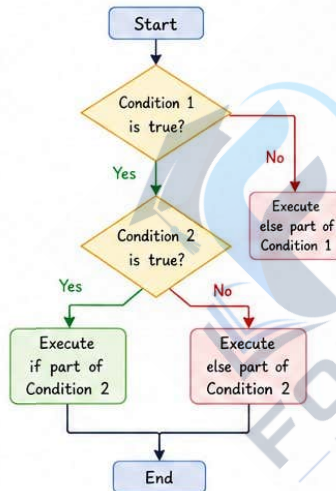
```
int age = 20;  
if (age >= 18) {  
    System.out.println("Eligible to vote");  
}  
else {  
    System.out.println("Not eligible to vote");  
}
```

Output: Eligible to vote

2. Nesting of if...else statement

An **if...else** statement can be placed inside another **if...else** statement.

Flowchart:



Syntax:

```
if (condition1) {  
    if (condition2) {  
        // statements  
    }  
    else {  
        // statements  
    }  
}  
else {  
    // statements  
}
```

Example:

```
public class NestedIfExample {  
    public static void main(String[] args) {  
        int marks = 75;  
        if (marks >= 50) {  
            if (marks >= 75) {  
                System.out.println("Grade: Distinction");  
            }  
            else {  
                System.out.println("Grade: First Class");  
            }  
        }  
        else {  
            System.out.println("Grade: Fail");  
        }  
    }  
}
```

How it works:

- First, **condition1** is checked.
- If **condition1** is **true**, then **condition2** is checked.
 - If **condition2** is **true** → **if** part of **condition2** runs.
 - If **condition2** is **false** → **else** part of **condition2** runs.
- If **condition1** is **false** → **else** part of **condition1** runs.

Output for above example:

Grade: Distinction

★ Note:

Nesting can be multiple levels deep for complex decisions.



Decision Making and Branching in Java

What is else...if ladder?

It is used to check multiple conditions one by one.
As soon as one condition is **true**, its block is executed and the rest are **skipped**.

Else...if ladder

Used when we have multiple conditions to check.

1. Syntax:

```
if (condition1) {  
    // statements  
}  
else if (condition2) {  
    // statements  
}  
else if (condition3) {  
    // statements  
}  
.  
.  
.  
else {  
    // statements  
}
```

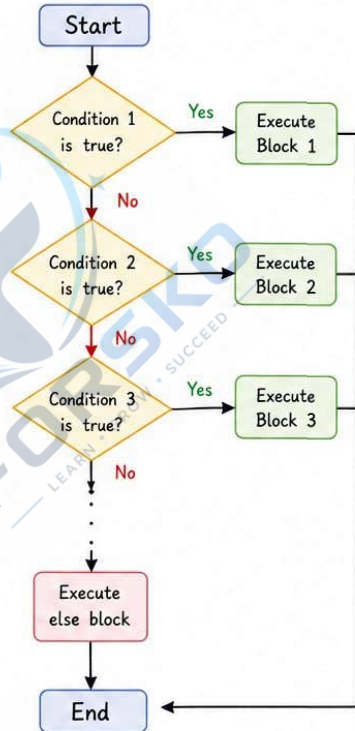
You can have any number of else if blocks.

else part is optional.

Example Use:

Used in grade calculation, fee slabs, age categories, discounts, etc.

2. Flowchart:



3. Example Program:

```
import java.util.Scanner;  
  
public class ElseIfLadderExample {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int marks;  
        System.out.print("Enter marks (0-100): ");  
        marks = sc.nextInt();  
  
        if (marks >= 90) {  
            System.out.println("Grade: A+");  
        }  
        else if (marks >= 75) {  
            System.out.println("Grade: A");  
        }  
        else if (marks >= 60) {  
            System.out.println("Grade: B");  
        }  
        else if (marks >= 40) {  
            System.out.println("Grade: C");  
        }  
        else {  
            System.out.println("Grade: Fail");  
        }  
  
        sc.close();  
    }  
}
```

4. Sample Output:

Enter marks (0-100): 92
Grade: A+

Enter marks (0-100): 68
Grade: B

Enter marks (0-100): 45
Grade: C

Enter marks (0-100): 35
Grade: Fail

5. Key Points:

- ★ Conditions are checked from top to bottom.
- ★ The first true condition's block is executed.
- ★ Once a block is executed, the rest are skipped.
- ★ else block runs only when no condition is true.

Decision Making and Branching in Java

Switch Statement

The **switch** statement is used to execute one block of code from multiple options based on the value of a variable/ expression.

1. Introduction

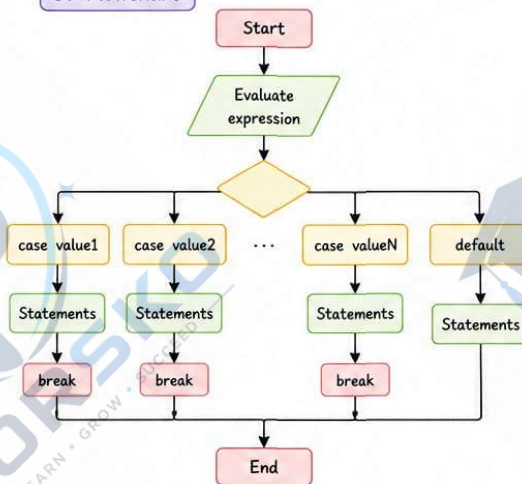
- Used when we have **multiple fixed values** to compare.
- It is more **readable** than multiple **else if** statements.
- Works with **byte, short, int, char, String, enum**.

2. Syntax

```
switch (expression) {  
    case value1:  
        // statements  
        break;  
    case value2:  
        // statements  
        break;  
    .  
    .  
    .  
    default:  
        // statements  
}
```

Optional
(default executes
if no case
matches)

3. Flowchart



4. Key Points

- ★ The expression is evaluated once.
- ★ The matching case block is executed.
- ★ **break** prevents the execution from "falling through" to next cases.
- ★ If **break** is omitted, the next case(s) will also execute.
- ★ Default block is optional.

5. Example Program

```
import java.util.Scanner;  
public class SwitchExample {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter day number (1-7): ");  
        int day = sc.nextInt();  
        String dayName;  
        switch (day) {  
            case 1: dayName = "Monday"; break;  
            case 2: dayName = "Tuesday"; break;  
            case 3: dayName = "Wednesday"; break;  
            case 4: dayName = "Thursday"; break;  
            case 5: dayName = "Friday"; break;  
            case 6: dayName = "Saturday"; break;  
            case 7: dayName = "Sunday"; break;  
            default: dayName = "Invalid day";  
        }  
        System.out.println("Day: " + dayName);  
        sc.close();  
    }  
}
```

6. Example Use

Used in menu-driven programs, calculator, month name finder, day name finder, grade calculator, etc.

7. Sample Output

Enter day number (1-7): 1
Day: Monday

Enter day number (1-7): 5
Day: Friday

Enter day number (1-7): 7
Day: Sunday

Enter day number (1-7): 10
Day: Invalid day

Decision Making and Looping: While statement

1. Introduction

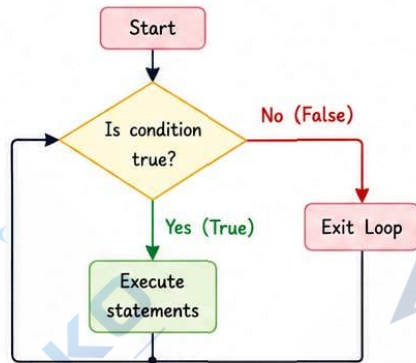
The **while** loop is an entry-controlled loop. It executes a block of code repeatedly as long as the given condition is **true**. When the condition becomes **false**, the loop terminates.

2. Syntax

```
while (condition) {  
    // statements  
}
```

Body of while loop executes repeatedly while condition is true.

3. Flowchart



4. Example Program

```
public class WhileExample {  
    public static void main(String[] args) {  
        int i = 1;  
        System.out.println("Numbers from 1 to 5:");  
        while (i <= 5) {           // condition  
            System.out.println(i); // loop body  
            i = i + 1;              // update  
        }  
        System.out.println("Loop finished.");  
    }  
}
```

5. How it works? (Step by Step)

1. Initialize $i = 1$
2. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 1, then i becomes 2.
3. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 2, then i becomes 3.
4. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 3, then i becomes 4.
5. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 4, then i becomes 5.
6. Check condition $i \leq 5 \rightarrow \text{True}$
→ Execute statements → Print 5, then i becomes 6.
7. Check condition $i \leq 5 \rightarrow \text{False}$
→ Exit loop.

6. Sample Output

Numbers from 1 to 5:

1
2
3
4
5

Loop finished.

Note:

The loop runs as long as the condition is true.

7. Example Use

- Reading input until user enters a valid value
- Processing items in a list until a condition is met
- Repeating a task until a specific goal is achieved

8. Key Points

- ★ The **while** loop checks the condition before executing the body.
- ★ If the condition is **false** at the beginning, the loop body will not execute at all.
- ★ Don't forget to update the variable inside the loop; otherwise, it may lead to an **infinite loop**.



Decision Making and Looping in Java

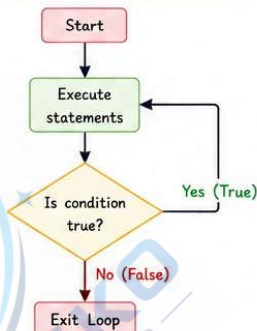
1. Do-While statement

The **do-while** loop is an **exit-controlled** loop. It executes the block of code **at least once**, and then repeats as long as the condition is **true**.

2. Syntax

```
do {  
    // statements  
} while (condition);
```

3. Flowchart



4. Example Program

```
public class DoWhileExample {  
    public static void main(String[] args) {  
        int i = 1;  
        System.out.println("Numbers from 1 to 5:");  
        do {  
            System.out.println(i);  
            i = i + 1;  
        } while (i <= 5);  
        System.out.println("Loop finished.");  
    }  
}
```

6. Key Points

- ★ The statements inside **do** block execute first, then the condition is checked.
- ★ The loop body executes **at least once** even if the condition is false initially.
- ★ Use **do-while** when you want the code to run once before checking the condition.



7. Example Use

- Menu driven programs
- User input validation
- Repeating a task until valid data is entered

8. Do-While vs For

Feature	Do-While	For
Type	Exit-controlled loop	Entry-controlled loop
When condition is checked	After executing the body	Before executing the body
Minimum executions	At least once	May execute zero times
Best used when	Iterations are not known, but at least one execution is required	Number of iterations is known in advance

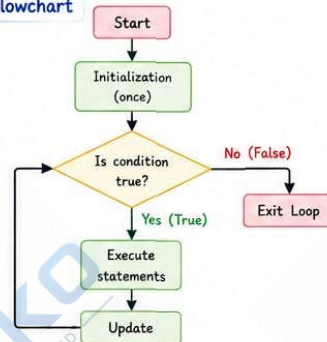
1. For statement

The **for** loop is an **entry-controlled** loop. It is used when the number of iterations (repetitions) is **known in advance**.

2. Syntax

```
for (initialization; condition; update) {  
    // statements  
}
```

3. Flowchart



4. Example Program

```
public class ForExample {  
    public static void main(String[] args) {  
        System.out.println("Numbers from 1 to 5:");  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(i);  
        }  
        System.out.println("Loop finished.");  
    }  
}
```

5. Sample Output

```
Numbers from 1 to 5:  
1  
2  
3  
4  
5  
Loop finished.
```

6. Key Points

- ★ Initialization runs once at the beginning.
- ★ Condition is checked **before** each iteration.
- ★ If condition is false initially, the loop body will not execute at all.
- ★ Update is executed after each iteration.



7. Example Use

- When number of iterations is known.
- Traversing arrays, lists, strings.
- Tables (e.g., multiplication table).
- Counting, summing, and processing a range of values.

Remember:

Choose the loop that best fits the logic and requirement of your program.



Decision Making and Looping in Java

Jump in Loop

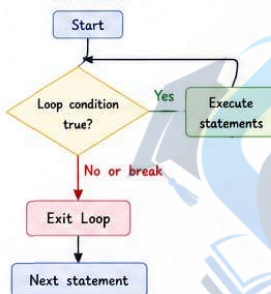
1. break Statement

The **break** statement is used to come out of the loop immediately, even if the loop condition is still true. After **break**, the program continues with the statement immediately after the loop.

Syntax:

```
break;
```

Flowchart:



Example Program:

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 6) { // exit when i becomes 6  
                break;  
            }  
            System.out.print(i + " ");  
        }  
        System.out.println("\nLoop terminated.");  
    }  
}
```

Output:

```
1 2 3 4 5  
Loop terminated.
```

Key Points:

- ★ **break** terminates the loop immediately.
- ★ Control comes out of the loop.

In loops, sometimes we need to skip the current iteration or terminate the loop immediately, Java provides two keywords for this purpose: **break** and **continue**.

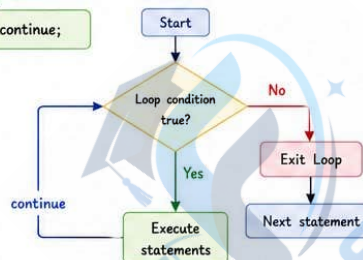
2. continue Statement

The **continue** statement is used to skip the current iteration and proceed to the next iteration of the loop. The remaining statements in the current iteration are not executed.

Syntax:

```
continue;
```

Flowchart:



Example Program:

```
public class ContinueExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i % 2 == 0) { // skip even numbers  
                continue;  
            }  
            System.out.print(i + " ");  
        }  
        System.out.println("\nLoop finished.");  
    }  
}
```

Output:

```
1 3 5 7 9  
Loop finished.
```

Key Points:

- ★ **continue** skips the current iteration.
- ★ Control remains in the loop for the next iteration.

4. Comparison

Feature	break	continue
Purpose	Terminates the loop immediately	Skips current iteration and continues
Where control goes	Comes out of the loop	Remains in the loop
Remaining statements in current iteration	Not executed	Not executed
Use when	You want to stop the loop completely	You want to skip some iterations

5. Notes

- ✓ **break** and **continue** can be used in for, while, and do-while loops.
- ✓ They affect only the innermost loop in case of nested loops.
- ✓ Use them carefully to make the program logic clear and efficient.



Remember:

break is like saying "Stop the loop!"
continue is like saying "Skip this turn, I'll try next time!"



3. Using break and continue in While Loop

Break Example:

```
int i = 1;  
while (i <= 10) {  
    if (i == 7) break; // exit loop  
    System.out.print(i + " ");  
    i++;  
}  
// Output: 1 2 3 4 5 6
```

Continue Example:

```
int i = 1;  
while (i <= 10) {  
    if (i % 2 == 0) { i++; continue; }  
    System.out.print(i + " ");  
    i++;  
}  
// Output: 1 3 5 7 9
```

6. Example: Using both in same loop

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue; // skip even numbers  
    if (i > 7) break; // exit when i > 7  
    System.out.print(i + " ");  
}  
// Output: 1 3 5 7
```

How it works:

- i = 1 → print
- i = 2 → even → continue
- i = 3 → print
- i = 4 → even → continue
- i = 5 → print
- i = 6 → even → continue
- i = 7 → print
- i = 8 → > 7 → break → exit loop